Raw decoder excerpt (continued) *preprocessing/pingstation_raw_decoder.py*

```
            out["verVelocityCMS"] = int(round(vs_fpm * 0.00508 * 100))

        if flags & self._F_SQUAWK:
            if idx + 2 > len(frame_bytes):
                return None
            out["squawk"] = int.from_bytes(frame_bytes[idx:idx + 2], "little", signed=False)
            idx += 2

        # Packed compressed flags byte (squawk changed / emergency / ident / on-ground).
        if flags & self._F_COMP_FLAGS:
            if idx + 1 > len(frame_bytes):
                return None
            out["compressedFlags"] = frame_bytes[idx]
            idx += 1

        # Minimum sanity: must at least resolve callsign or position/speed info.
        if len(out) == 1:
            return None
        return out

    def _decode_lat_lon(self, frame_bytes, prev_lat=None, prev_lon=None):
        candidates = []

        # Observed Pingstation layout in sample logs:
        # latitude/longitude as little-endian float32 at offsets 25..28 and 29..32.
        if len(frame_bytes) >= 33:
            try:
                lat_le = struct.unpack("<f", frame_bytes[25:29])[0]
                lon_le = struct.unpack("<f", frame_bytes[29:33])[0]
                if -90.0 <= lat_le <= 90.0 and -180.0 <= lon_le <= 180.0:
                    candidates.append(("le_25", lat_le, lon_le))
            except struct.error:
                pass

        # Legacy fallback used by older parser assumptions.
        if len(frame_bytes) >= 36:
            try:
                lat_be = struct.unpack(">f", frame_bytes[28:32])[0]
                lon_be = struct.unpack(">f", frame_bytes[32:36])[0]
                if -90.0 <= lat_be <= 90.0 and -180.0 <= lon_be <= 180.0:
                    candidates.append(("be_28", lat_be, lon_be))
            except struct.error:
                pass

        if not candidates:
            return None, None

        has_prev = (
            prev_lat is not None
            and prev_lon is not None
            and prev_lat != 0
            and prev_lon != 0
        )
        if has_prev:
            # Keep trajectory continuous when multiple layouts look numerically valid.
            def continuity_score(item):
                _, lat, lon = item
                return abs(lat - prev_lat) + abs(lon - prev_lon)

            _, lat, lon = min(candidates, key=continuity_score)
            return lat, lon

        # Prefer observed Pingstation sample layout for first fix.
        for layout_name in ("le_25", "be_28"):
            for name, lat, lon in candidates:
                if name == layout_name:
                    return lat, lon

        _, lat, lon = candidates[0]
        return lat, lon

    def decode_frame(self, frame_bytes, source_ip=None, timestamp=None, callsign_hint=None):
        # Control/housekeeping packets are usually shorter and not useful for aircraft state.
        if len(frame_bytes) < 23:
```